RESUMEN DE PAPER · GENERACIÓN PROCEDURAL DE NIVELES 2D

Patrones como objetivos para la generación de niveles

Patterns as Objectives for Level Generation

Autores	Steve Dahlskog (Malmö University) y Julian Togelius (IT University of Copenhagen)
Publicación	Proceedings of the Second Workshop on Design Patterns in Games (DPG), ACM, 2013
Palabras clave	Patrones de diseño de juegos, generación procedural de contenido, Super Mario Bros., algoritmos genéticos
Técnica central	Algoritmo evolutivo sobre cadenas de “rebanadas verticales”; la aptitud cuenta patrones de diseño encontrados

Resumen

El paper discute cómo usar **patrones de diseño** en la generación procedural de niveles, tomando como caso el clásico *Super Mario Bros.* (SMB). En un trabajo anterior, los autores analizaron los niveles del juego, identificaron patrones recurrentes y construyeron un generador que simplemente *concatenaba* esos patrones. Aquí invierten el enfoque: los patrones pasan a ser **objetivos de diseño** y no piezas de construcción. Implementan esta idea con computación evolutiva: los niveles se representan como secuencias de **rebanadas verticales** tomadas del juego original, y la función de aptitud cuenta cuántos patrones aparecen. Se hace un análisis cualitativo y un pequeño estudio con jugadores para identificar fortalezas y desafíos.

1. Introducción

El trabajo combina patrones de diseño y PCG en el diseño de niveles de plataformas 2D, en particular SMB. Parte de un análisis previo que encontró **23 patrones** agrupados en **5 familias**: Enemigos, Huecos, Valles, Caminos múltiples y Escaleras. El generador previo concatenaba esos patrones variando altura, longitud o dificultad, con poca variedad.

La propuesta nueva separa conceptualmente **los bloques de construcción** de **los objetivos de diseño**: el material de construcción son piezas de 1 tile de ancho extraídas de SMB (rebanadas verticales) y los objetivos son los patrones. Así se consigue mucha más variedad usando solo material existente, y niveles que se parecen a SMB tanto a nivel micro (las rebanadas) como macro (los patrones).

2. Antecedentes

2.1 Patrones de diseño

La idea viene de la arquitectura: Alexander et al. (años 70) crearon un lenguaje de patrones donde cada patrón describe un problema recurrente y el núcleo de su solución. Luego pasó al software orientado a objetos (Gamma et al., “Gang of Four”).

2.1.1 Patrones de diseño y juegos

Los patrones en juegos digitales sirven como apoyo creativo y de comunicación entre diseñadores, como herramienta de análisis y aprendizaje, y para entender el comportamiento de jugadores en playtesting. Björk y Holopainen los documentaron extensamente, y se han estudiado para mecánicas,

juegos de acción clásicos, RPGs, FPSs y juegos sociales tipo “'Ville”.

2.2 Contenido de juego y desarrollo

Un juego puede verse como la unión de un **motor** (entrada, IA, presentación) y el **contenido** (mallas, texturas, geometría de niveles, personajes, misiones, ítems).

2.2.1 Factores de costo

Desarrollar juegos es cada vez más caro, principalmente porque plataformas más potentes exigen más contenido y de mejor calidad, lo que a su vez complejiza los motores.

2.2.2 Desarrollo de juegos y PCG

La PCG se ha usado para ahorrar tiempo (*Darwinia*), dinero (*Just Cause*), explorar contenido (*The Sentinel*), dar variedad (*Minecraft*) o ahorrar memoria (*Elite*). Los autores distinguen tres formas de combinar PCG y diseño: (1) tareas acotadas y bien probadas (mapas de *Civilization*, armas de *Borderlands*); (2) PCG como proveedor central de contenido (*Minecraft*, *Dwarf Fortress*, *FTL*…); (3) partir de contenido hecho a mano y que la PCG emule las decisiones del diseñador con variación (online u offline, p. ej. para expansiones).

2.3 Encajando en el patrón

Los diseñadores de niveles disponen componentes en un editor y los ajustan al tema del juego. Para replicar eso con PCG, los autores aplicaron antes un análisis de contenido (heurísticas + grupos de ritmo) que dio un conjunto de patrones de SMB y un prototipo que elegía patrones al azar, los modificaba y los ponía en secuencia: muy parecido al original, pero con poca variación (solo por parametrización). Aquí los patrones se usan como **objetivo del método evolutivo**, para dar mayor “libertad creativa” (un espacio de diseño más grande, aunque más acotado que la aleatoriedad pura). La meta: recrear el *look-and-feel* de SMB y aun así ofrecer niveles nuevos.

2.3.1 Ejemplos de patrones

Tabla 1. Ejemplos de patrones de Super Mario Bros.

Familia	Patrón	Descripción
Enemigos	Enemy	Un enemigo solo.
	2-Horde	Dos enemigos juntos.
	3-Horde	Tres enemigos juntos.
	4-Horde	Cuatro enemigos juntos.
	Roof	Enemigos bajo una plataforma colgante que hace rebotar a Mario en el “techo”.
Caminos múltiples	2-Path	Una plataforma colgante que permite elegir entre caminos.
	3-Path	Dos plataformas colgantes que permiten elegir entre caminos.
	Risk and Reward	Camino múltiple donde uno tiene recompensa, pero con un hueco o enemigo que lo vuelve riesgoso.

2.4 Trabajo relacionado

- **Tanagra** (Smith et al.): herramienta de iniciativa mixta que usa un *solver* de restricciones para crear geometría según patrones de un beat (hueco, resorte) y compuestos (valle, meseta).
- **Occupancy-Regulated Extension** (Mawhorter y Mateas): agrega piezas tipo rompecabezas de forma composicional.
- Patrones como objetivos en PCG basada en búsqueda: la evaluación de **puntos de estrangulamiento** (*choke points*) al evolucionar mapas de *StarCraft*.

3. Mario

El SMB original (Nintendo, 1985) inspiró el clon de dominio público de Markus “Notch” Persson, base de las competencias Mario AI y de PCG. Tiene 8 mundos de 4 niveles; el 4.º de cada mundo es un nivel de jefe con distinta disposición.

3.1 La representación original

La implementación del cartucho de NES está optimizada para espacio, no para legibilidad: cada nivel es una cadena de “páginas” y cada objeto se describe con valores hexadecimales y posición dentro de la página (W1L1 tiene 13 páginas). Es eficiente para líneas horizontales/verticales de bloques, tuberías, escaleras de roca y astas; algunos objetos tienen doble función (cambiando unos pocos valores, un nivel submarino se vuelve terrestre). Los autores sugieren que esta optimización de memoria pudo influir en el diseño de SMB, igual que las limitaciones de la Atari 2600 influyeron en sus juegos: dibujar más objetos cuesta más que alargar una sección.

4. Representación y mapeo genotipo–fenotipo

Se exploraron tres representaciones, dado que en optimización estocástica la relación genotipo (sobre lo que actúa el algoritmo) – fenotipo (lo que se evalúa) es crítica:

- **ρ0**: matriz 2D de enteros por bloque. Espacio de búsqueda enorme con muy pocos niveles jugables; descartada.
- **ρ1**: alturas enteras de 0 (hueco) a 10 (obstáculo máximo). Producía pocos obstáculos en el suelo y no incluía enemigos; añadir tipos de enemigo × alturas × tipos de geometría hacía explotar el espacio (costoso, sobre todo para generación en línea).
- **ρ2** (adoptada): **rebanadas verticales**, descrita a continuación.

4.1 Rebanadas verticales

La idea es mirar el contenido desde la perspectiva de *Mario*, que avanza paso a paso saltando sobre objetos a distintas alturas. Así, el contenido de SMB se ve como columnas que, juntas, forman los patrones. El **genotipo** es una cadena de **200 símbolos** de un alfabeto de **24**; cada símbolo es una rebanada de 1 bloque de ancho y 13 de alto, tomada de patrones del SMB original (sobre todo W1L1 y W1L2). El **fenotipo** se obtiene concatenando las rebanadas, así que todos los niveles miden 200 bloques.

Se temía tener que programar un chequeo de compatibilidad entre rebanadas, pero el análisis mostró que la variación de rebanadas en SMB es sorprendentemente limitada. Por ejemplo, la sección de la Fig. 1 (un 2-Path de W1L1) son solo dos tipos de rebanada (una usada tres veces, otra una vez), más una rebanada simple de suelo para separar instancias.

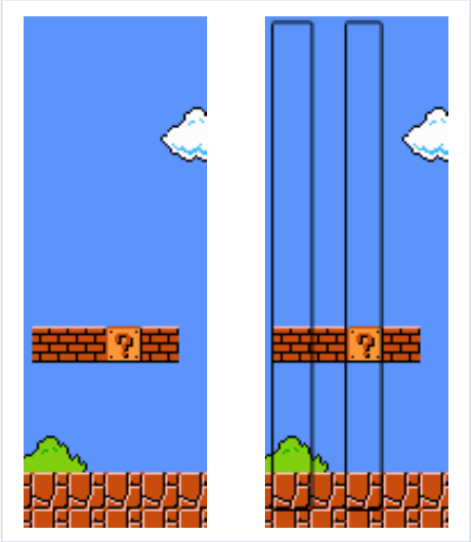


Figura 1. Instancia simple del patrón 2-Path en SMB (izquierda), reproducible con solo 2 rebanadas verticales (marcadas con marcos negros, derecha).

4.2 Uniendo las piezas

Ejemplo con el patrón **Enemy: 3-Horde** (Fig. 2): se describe como tres rebanadas idénticas, cada una con un tile de suelo y un enemigo (Fig. 3).

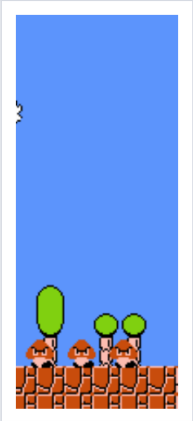


Figura 2. Un patrón 3-Horde “en estado salvaje” (SMB, Mundo 8 Nivel 1).

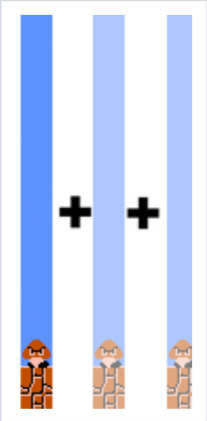


Figura 3. Agregando rebanadas verticales para formar una instancia del patrón de la Fig. 2.

4.2.1 Ejemplo 1

Un 2-Path se reproduce con 2 rebanadas: una con un ladrillo y otra con un bloque “?” (Fig. 1).

4.2.2 Ejemplo 2

Agregando una rebanada con dos bloques (ladrillo y “?”) y reutilizando las dos de la Fig. 1 se obtiene un 3-Path (Fig. 4).

4.2.3 Ejemplo 3

Agregando una rebanada con dos bloques “?” y reutilizando una de la Fig. 1 se obtiene otro 3-Path (Fig. 5).



Figura 4. Un patrón 3-Path.



Figura 5. Otro patrón 3-Path.

5. Función de aptitud

El generador sigue el ciclo de la Fig. 6. La aptitud recorre cada individuo y le asigna un valor: **más alto cuanto más complejo (o improbable al azar) es el patrón** que contiene (Tabla 2). Algunos patrones solo se premian "por apilamiento": se suman el comienzo y el final de patrones hasta un valor comparable a "medio–alto". Por ejemplo, tres Goombas consecutivos reciben un valor positivo; una secuencia de rocas de altura creciente recibe más cuanto más larga es.

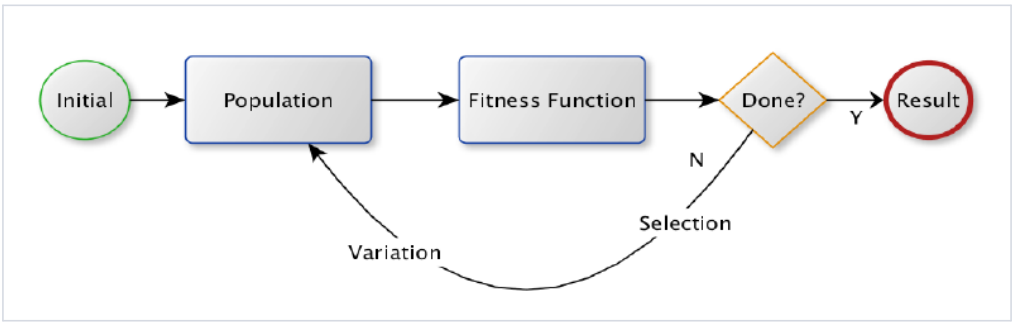


Figura 6. Ejecución principal del generador: Inicial → Población → Función de aptitud → ¿Terminado? (sí: Resultado; no: Selección y Variación).

Tabla 2. Patrones soportados en la función de aptitud y su recompensa.

Familia	Patrón	Recompensa
Enemigos	Enemy · 2-Horde · 3-Horde · 4-Horde	Baja
	Roof	Media
Huecos	Gaps	Baja
	Multiple gaps · Variable gaps	Por apilamiento
	Gap enemy	Baja–Media por apilamiento
	Pillar gap	Pilar alta
Valles	Valley	Baja
	Pipe valley	Media
	Empty valley · Enemy valley · Roof valley	Por apilamiento
Caminos múltiples	2-Path · 3-Path	Media–Alta
	Risk and Reward	Por apilamiento
Escaleras	Stair up · Stair down · Empty stair valley	Baja
	Enemy stair valley · Gap stair valley	Por apilamiento

Otras reglas: las secuencias **no jugables** reciben un valor muy negativo (pero no $-\infty$), para que aún puedan reproducirse y conservar sus partes buenas vía mutación o cruce; las secuencias **poco interesantes** reciben un valor levemente negativo, aunque se toleran algunas (p. ej. suelo plano) para mantener una expresión tipo **beat** (alternancia entre partes intensas y calmas). Los comienzos y finales de patrones dan menos que un patrón completo, pero comienzo + patrón + final se acumula, lo que permite el **apilamiento de patrones** que faltaba en el prototipo anterior.

6. Algoritmo evolutivo

- Población de **200** niveles, inicializados como cadenas uniformemente aleatorias del alfabeto de 24 rebanadas.
- Estrategia evolutiva simple **μ+λ** (los autores indican $\mu = \lambda = 50$) con mutación y cruce de un punto.
- En cada generación: evaluar, ordenar por aptitud, conservar el 50 % superior y descartar el resto; los sobrevivientes se cruzan por pares en orden de ranking (1.º con 2.º, etc.) para rellenar la población.
- **Cruce de un punto** con el punto fijo en la mitad de la cadena (Fig. 7).
- **Mutación**: inyectar un símbolo aleatorio en una posición aleatoria, para evitar óptimos locales.
- Como se ejecuta *offline*, la búsqueda corre **10 000 generaciones**.

Nota del resumen: el texto mezcla la notación $\mu = \lambda = 50$ con una población de 200 en la que se conserva la mitad; el procedimiento operativo descrito es el de truncamiento al 50 %.

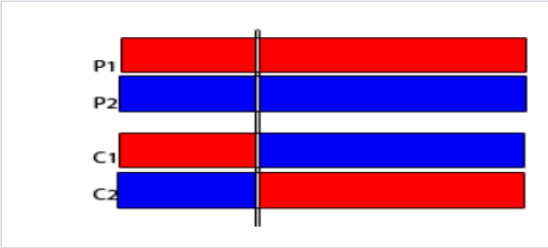


Figura 7. Cruce de un punto: el padre 1 (rojo) y el padre 2 (azul) generan los hijos 1 y 2 con colores mezclados.

7. Ejemplos de niveles generados

El enfoque evolutivo con rebanadas produce niveles con patrones, patrones apilados y parecido al original (Figs. 8–11). Sin embargo, la implementación actual no soporta bien el concepto de **beats** (alternar intensidad alta y baja): tiende a apilar patrones muy juntos y a llenar en exceso el espacio. La versión invertida (α, Fig. 8) llena aún más el espacio que el generador propuesto.

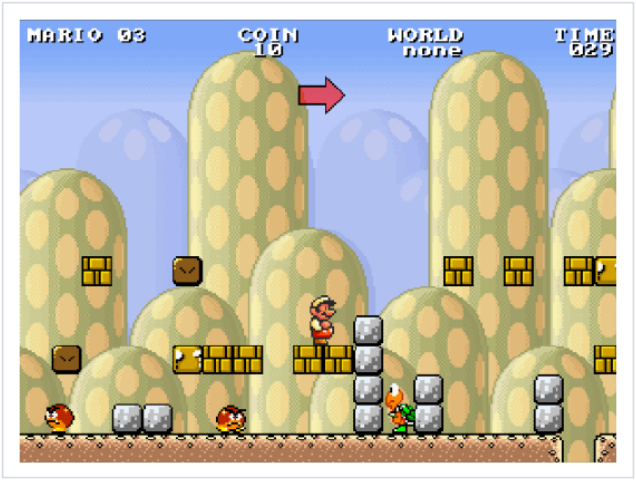


Figura 8. Nivel α: tendencia a sobrellenar el nivel.

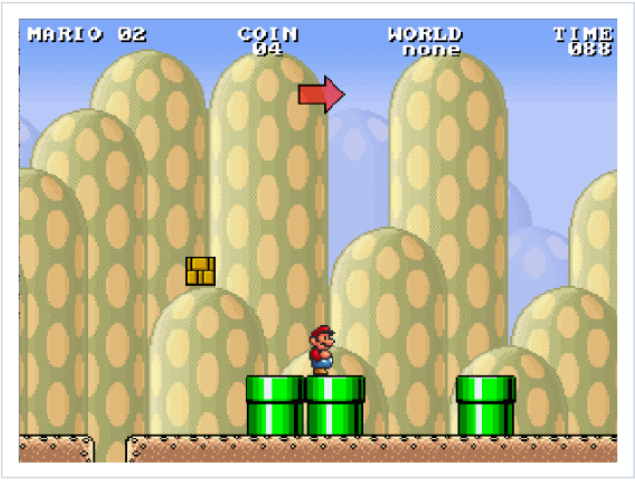


Figura 9. Nivel β: tendencia a apilar patrones.



Figura 10. β a veces apila patrones demasiado juntos.

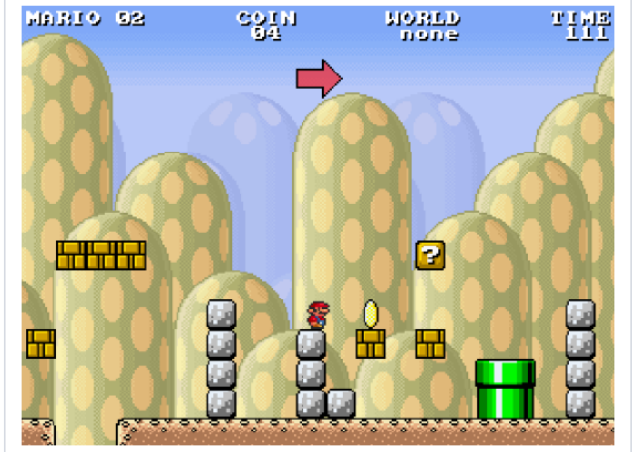


Figura 11. β casi sobrellena el espacio de juego, como α.

8. Evaluación

Se hizo un *playtest* simple con tres niveles generados desde tres enfoques:

- **α**: función de aptitud *invertida* (castiga cualquier patrón o comienzo/fin de patrón).
- **β**: la función de aptitud propuesta.
- **γ**: combinación del prototipo basado en patrones anterior con contenido que imita el original.

Participaron **24 jugadores experimentados** (23 hombres, 1 mujer, veinteañeros) en PCs de gama alta con teclado. Para controlar el sesgo de orden, tres grupos de 8 jugaron en órdenes distintos (α-β-γ; β-α-γ; γ-β-α). Además de una pregunta abierta, calificaron cada nivel en escala de 6 puntos en: (1) Aburrido–Divertido, (2) Nada similar–Similar al original y (3) Fácil–Difícil.

Los resultados muestran diferencias **marginales**: β pareció algo más divertido que los demás, más similar al original que α y más fácil de completar que γ (Tabla 3, Fig. 12).

Tabla 3. Resultados por nivel (escala 1–6). Orden de filas por versión: Divertido, Similar, Difícil.

Versión	Pregunta	Promedio	Mediana	Desv. estándar
α	Divertido	3,75	4	1,041
α	Similar	4,125	4	1,062
α	Difícil	3,5	3	0,791
β	Divertido	**3,792**	4	1,159
β	Similar	4,625	5	0,906
β	Difícil	2,375	2	0,989
γ	Divertido	3,708	4	0,923
γ	Similar	**4,833**	5	0,875
γ	Difícil	3,292	3	1,006

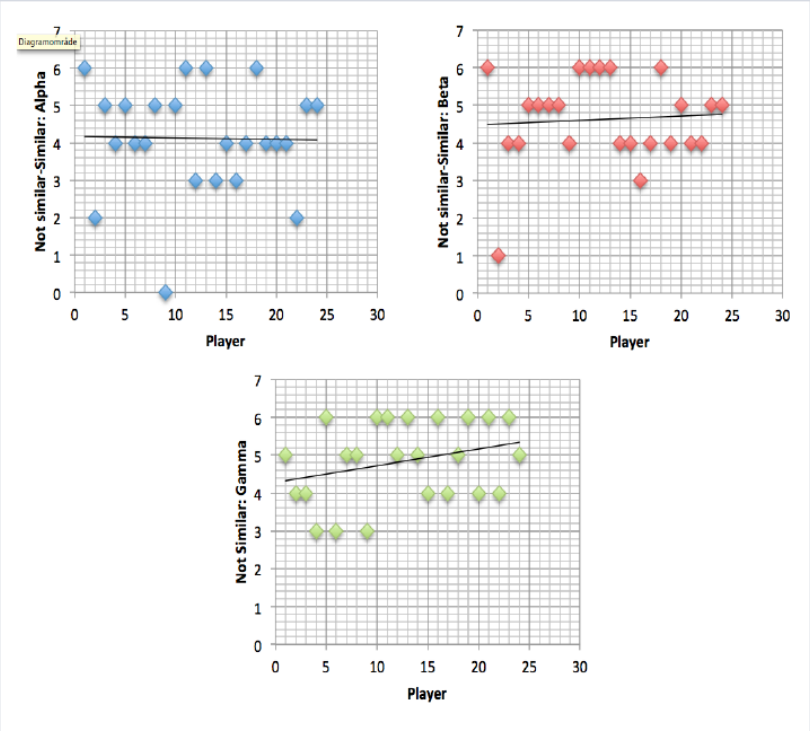


Figura 12. Respuestas “Nada similar–Similar” por jugador: azul = α, rojo = β, verde = γ.

9. Discusión

Los enfoques evolutivos funcionan bien con patrones en cuanto a variación y permiten **apilar patrones**. Pero afinar la función de aptitud es delicado: al introducir patrones nuevos, sus recompensas o castigos pueden alterar configuraciones previas. Los autores sugieren aplicar otros métodos de análisis de contenido cuando se usan bloques más pequeños que beats o patrones, para lograr frecuencias adecuadas de recompensas, enemigos y geometría. Como cada mundo de SMB tiene un *look-and-feel* distinto, planean un **análisis de frecuencias** del contenido original para crear funciones de aptitud específicas por mundo y nivel.

10. Conclusiones

El paper muestra cómo los patrones pueden usarse como **objetivos** en PCG en vez de bloques de partida, con una implementación para SMB. Los niveles originales reaparecen de dos formas: como **rebanadas verticales** de grano fino recombinadas en la representación evolucionable, y como **patrones** de alto nivel que guían la búsqueda. Así, los niveles evolucionados conservan buena parte del aspecto de Mario y el generador puede producir una amplia variedad. El estudio exploratorio sugiere que los niveles que maximizan patrones se perciben más similares al juego original.

Ideas clave

- **Inversión conceptual:** patrones = objetivos de la búsqueda, rebanadas = material de construcción.
- **Representación compacta:** 24 rebanadas de 1×13 bastan para expresar los patrones de SMB.
- **Limitación:** no modela bien los beats (descanso vs. intensidad) → niveles sobrecargados.
- **Limitación:** la aptitud es un sistema de pesos manual, frágil al añadir patrones; evaluación con muestra pequeña y diferencias marginales.